RAJIV GANDHI INSTITUTE OF PETROLEUM TECHNOLOGY, JAIS, AMETHI

Department of Computer Science and Engineering



B.Tech. 2nd Year

Object Oriented Methodologies (CS224)

By

Dr. Kalka Dubey

UNIT-IV (Standard Template Library (STL) of C++)

- Friend Function
- Inline and non-inline member functions of C++ classes
- Study of Various Files and Streams
- Creating file
- File Manipulation
- C++ Standard Library
- Template classes
- High-level view of modules

Friend Functions/Classes

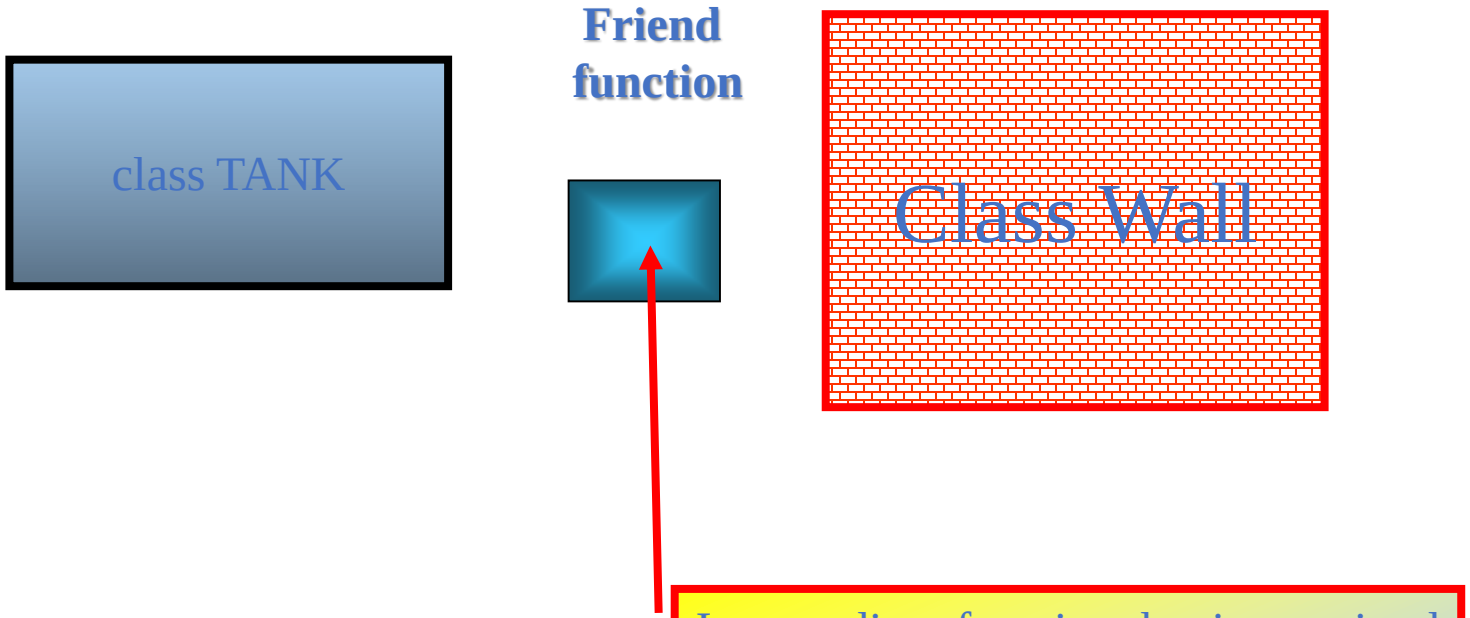
friends keywords allow functions/classes access to the **private data of other classes.**

Friend Functions/Classes

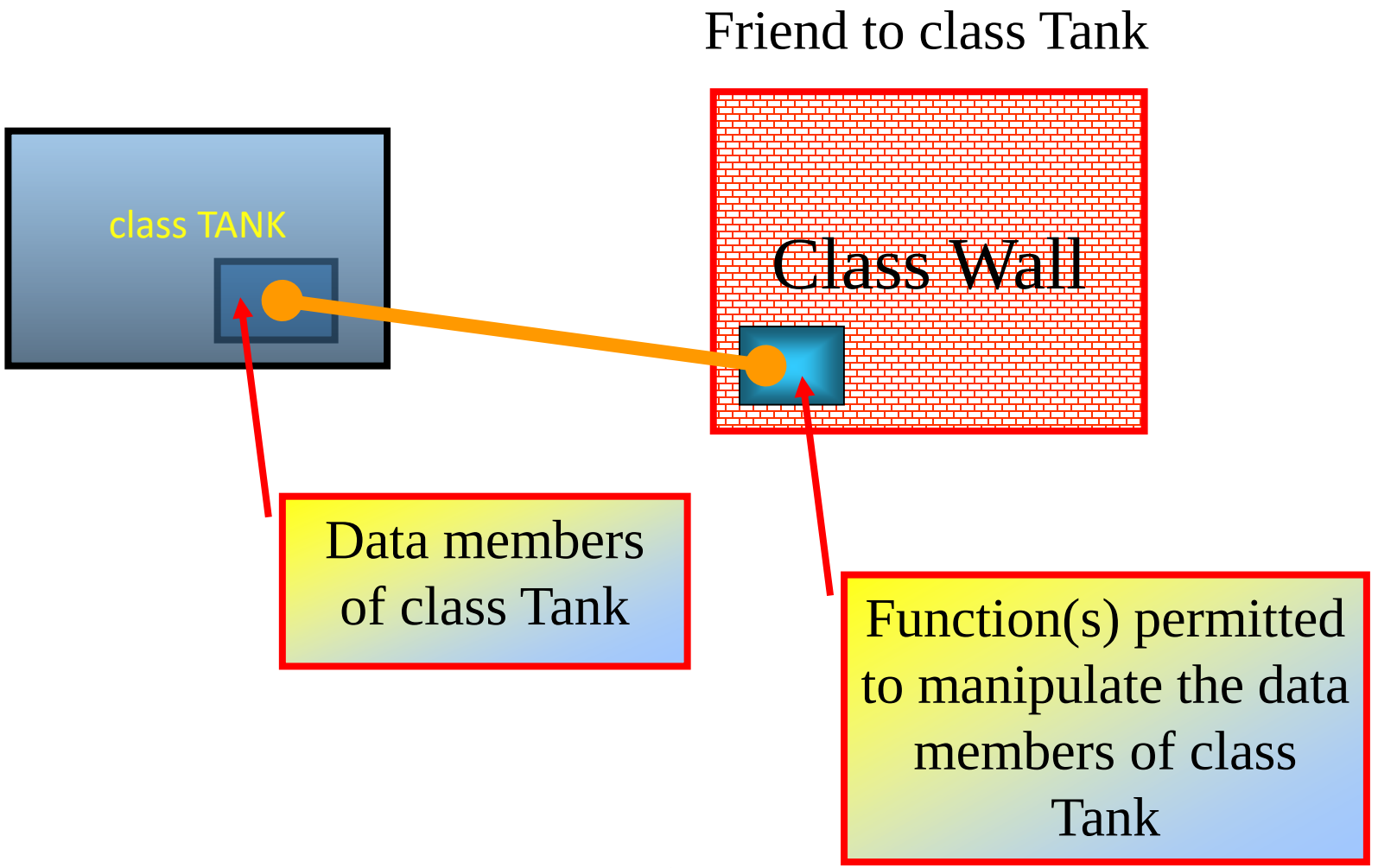
- A '**friend**' function has access to all 'private' members of the class for which it is a 'friend'.

- To declare a 'friend' function, include its prototype within the class, preceding it with the C++ keyword 'friend'.

Friend Functions/Classes



Friend Functions/Classes



Example 1:

```
#include <iostream>
using namespace std;
class Box
{
double width;
public:
void printWidth( Box box );
void setWidth( double wid );
};
// Member function definition
void Box::setWidth( double wid )
{
width = wid;
}
void printWidth( Box box )
{
cout << "Width of box : " << box.width
<<endl;
}
int main( )
{
Box box;
Box.width = 25.6;
box.setWidth(10.0);
printWidth( box );
return 0;
}
```

Output

Output : Error

'double Box::width' is private within this contex

Example 2: Friend Function

```
#include <iostream>
using namespace std;
class Box
{
double width;
public:
friend void printWidth( Box box );
void setWidth( double wid );
};
// Member function definition
void Box::setWidth( double wid )
{
width = wid;
}
void printWidth( Box box )
{
cout << "Width of box : " << box.width
<<endl;
}
int main( )
{
Box box;
box.setWidth(10.0);
printWidth( box );
return 0;
}
```

Output

Output :

Width of box : 10

Friend Functions/Classes

```
class T {
 public:
   friend void a();
   int m();
 private:  // ...
};

void a() {// can access
         // private data in T...}


class S {
 public:
   friend int T::m();
   //...
};

class X {
public:
   friend class T;
   //...
};
```

- function **a()** can access private data in **T**
- **m()** can access private data in **S**
- all functions of **T** can access private data in **X**
- **friends** should be used with caution: they by-pass C++'s data hiding principle.
- It is the responsibility of the code for which access is to be given to say who it's friends are - i.e. who does it trust!

Example 2: Friend Function

```
#include <iostream>
using namespace std;
class A
{
   int x;
   public:
   void setdata(int i)
   {
      x=i;
   }
 friend void min(A,B);
};
class B
{
   int y;
   public:
   void setdata(int i)
   {
      y=i;
   }
 friend void min(A,B);
};
void min(A a, B b)
{
   if(a.x<=b.y)
    cout << a.x << std::endl;
   else
    cout << b.y << std::endl;
}
  int main()
{
  A a;
  B b;
  a.setdata(10);
  b.setdata(20);
  min(a,b);
   return 0;
 }
```

Output

Output :

10

Example 3: Friend Class

```
#include <iostream>
using namespace std;
class A
{
   int x =5;
   friend class B;
};
class B
{
 public:
   void display(A a)
   {
      cout<<"value of x is : "<<a.x;
   }
};
int main()
{
   A a;
   B b;
   b.display(a);
   return 0;
}
```

Output

Output :

value of x is : 5

Overloading using friend function

- All the functions were part of the class but we can also overload all operators except few using friend function too.

- To overload a binary + we declare a function as a class member function as :

demo operator + (demo A)

- As the operator function is a member function of class left side operand must be an object of the class. This is must as internally d1+d2 is treated as d1.operator + (d2).

friend demo operator + (demo A, demo B);

Operator overloading using Friend

- There are operators which cannot be overloaded using friend function. They are given as :

S.No.	*Operators*	*Name of Operator*
1.	=	Assignment operator
2.	->	Pointer to member operator
3.	[ ]	Subscript operator
4.	( )	Function call operator.

Example 1: Overloading binary + operator using friend

```
#include <iostream.h>
using namespace std;
class demo
{
int num;
public :
demo( ){}
demo(int x)
{
num =x;
}
friend demo operator +(demo, int);
void show(char *s)
{
cout<<“num of object “<<s<<“=”<<num<<endl;
}
};
demo operator +(demo T, int x)
{
demo temp;
temp.num=T.num+x;
return temp;
}
void main( )
{
demo d1(100), d2;
d2=d1+50;
d1.show(“d1”);
d2.show(“d2”);
}
```

Output

Output :

num of object d1=100

num of object d2=150

Inline and non-inline member functions

- C++ programming language provides various important and fascinating methods and in-built functions to its users.

- When it comes to enhancing the performance of the program, the inline function proves to be an excellent concept.

- Inline function in C++ is an enhancement feature that improves the execution time and speed of the program.

- The main advantage of inline functions is that you can use them with C++ classes as well.

Inline and non-inline member functions

- When an instruction of a function call is encountered during the compilation of a program, its memory address is stored by the compiler.

- The function arguments are copied on the stack and after the execution of the code, the control is transferred to the calling instruction.

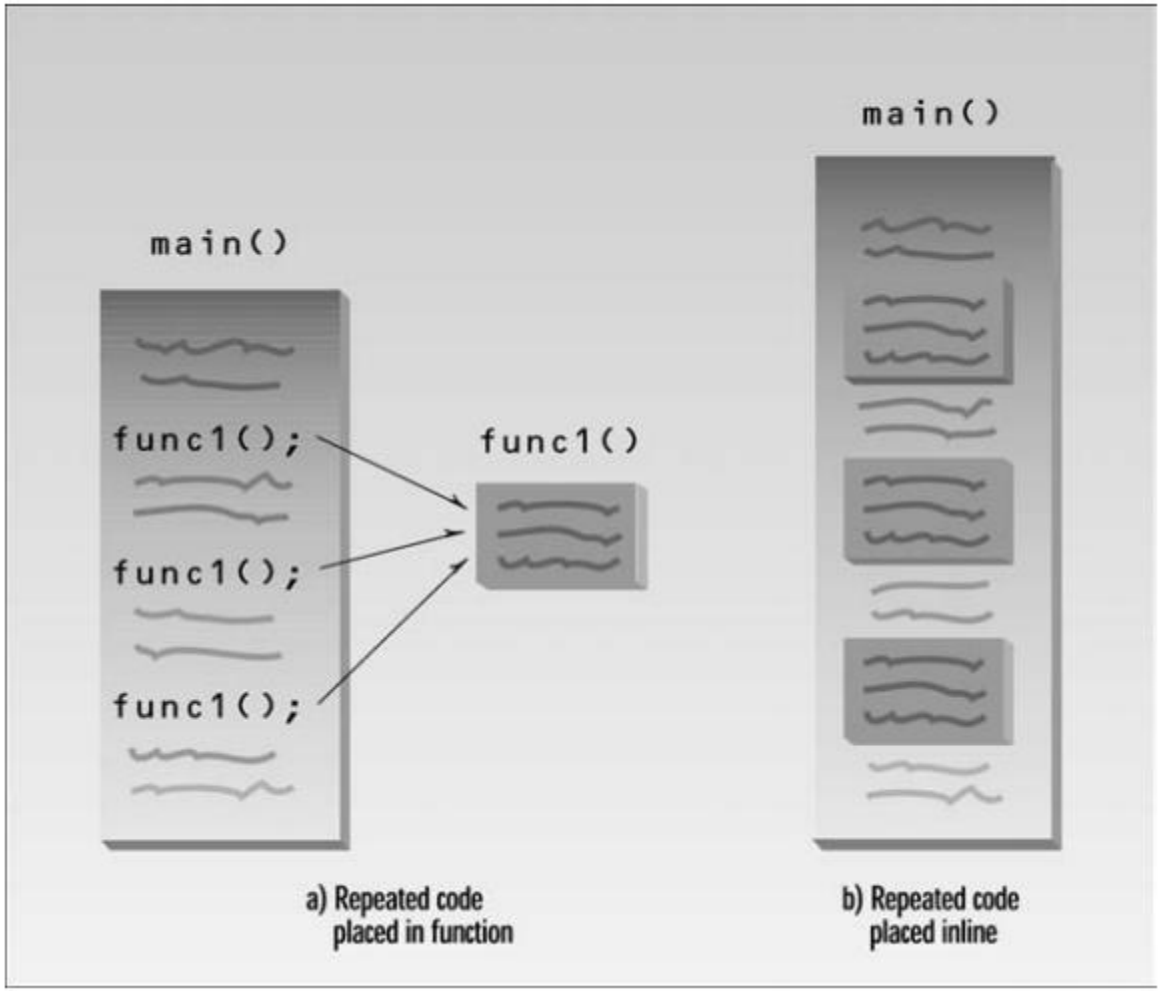
- This process can sometimes cause overhead in function calls, especially if the function is small and its execution time is less than the switching time.

Inline and non-inline member functions

- This issue is resolved by using the inline functions. These functions overcome the overhead and also make the program faster by reducing the execution time of the program.

- In case of inline functions, the compiler does not go through the above process of switching between the stack and calling function. Instead, it copies the definition of the inline function and the calling instruction with that definition.

Inline and non-inline member functions



Inline and non-inline member functions

- Following are some key points that you need to keep in mind while dealing with inline functions:

- Inline functions that are small have higher efficiency and better results than the lengthier inline functions. So, try to keep your inline functions small in length.

- Although these functions increase the efficiency of the program and improve its execution, you should not convert all the functions into inline functions. If you convert larger functions to inline, it may lead to code bloat and reduce the functioning quality of the program.

- Always try to define large functions outside the class, since functions defined inside a class are automatically defined as inline and this will affect the program negatively. You can use scope resolution (::) for this purpose.

Compiler Does Not Consider Inline Functions

- The compiler decides whether to accept or decline the request of inlining the function based on some circumstances as discussed below:

- The compiler denies the request to make a function an inline function if it contains a return statement but does not return anything.
- All those functions that contain a loop statement (for loop, while loop, or do-while loop) can not be considered as an inline function by the compiler.
- The compiler will not consider a function to be an inline function if it is recursive.
- If the function contains one or more static variables, the compiler will deny the request to make it an inline function.
- If the function contains a switch or a go-to statement, the compiler will deny the suggestion to make it an inline function.

Example

```
inline void printSum(int num1,int num2)
{
   cout << num1 + num2 << "\n";
}

int main()
{
    printSum(10, 20);

   printSum(2, 5);

   printSum(100, 400);

   return 0;

}
```

Example

```
inline void printSum(int num1, int num2){
    cout << num1 + num2 << "\n";
}
int main() {
    printSum(10, 20);
    printSum(2, 5);
    printSum(100, 400);
}
```

During Compilation

```
inline void printSum(int num1, int num2){
    cout << num1 + num2 << "\n";
}
int main() {
    cout << 10 + 20 << "\n";
    cout << 2 + 5 << "\n";
    cout << 100 + 400 << "\n";
}
```

Inline function and classes

- It is also possible to define the inline function inside the class.
- In fact, all the functions defined inside the class are implicitly inline.
- Thus, all the restrictions of inline functions are also applied here.
- If you need to explicitly declare inline function in the class then just declare the function inside the class and define it outside the class using inline keyword.

Inline function and classes

```
class S
{
public:
    inline int square(int s) // redundant use of inline
    {
        // this function is automatically inline
        // function body
    }
};
```

- The above style is considered as a bad programming style.

- The best programming style is to just write the prototype of function inside the class and specify it as an inline in the function definition.

Inline function and classes

```
class S
{
public:
    int square(int s); // declare the function
};

inline int S::square(int s) // use inline prefix
{

}
```

Inline function and classes

```
#include <iostream>
using namespace std;
class operation
{
    int a,b,add,sub,mul;
    float div;
public:
    void get();
    void sum();
    void difference();
    void product();
    void division();
};
inline void operation :: get()
{
    cout << "Enter first value:";
    cin >> a;
    cout << "Enter second value:";
    cin >> b;
}

inline void operation :: sum()
{
    add = a+b;
    cout << "Addition of two numbers: " << a+b <<
"\n";
}

inline void operation :: difference()
{
    sub = a-b;
    cout << "Difference of two numbers: " << a-b <<
"\n";
}
```

Inline function and classes

```
inline void operation :: product()
{
   mul = a*b;
   cout << "Product of two numbers: " << a*b << "\n";
}

inline void operation ::division()
{
   div=a/b;
   cout<<"Division of two numbers: "<<a/b<<"\n" ;
}

int main()
{
   cout << "Program using inline function\n";
   operation s;
   s.get();
   s.sum();
   s.difference();
   s.product();
   s.division();
   return 0;
}
```

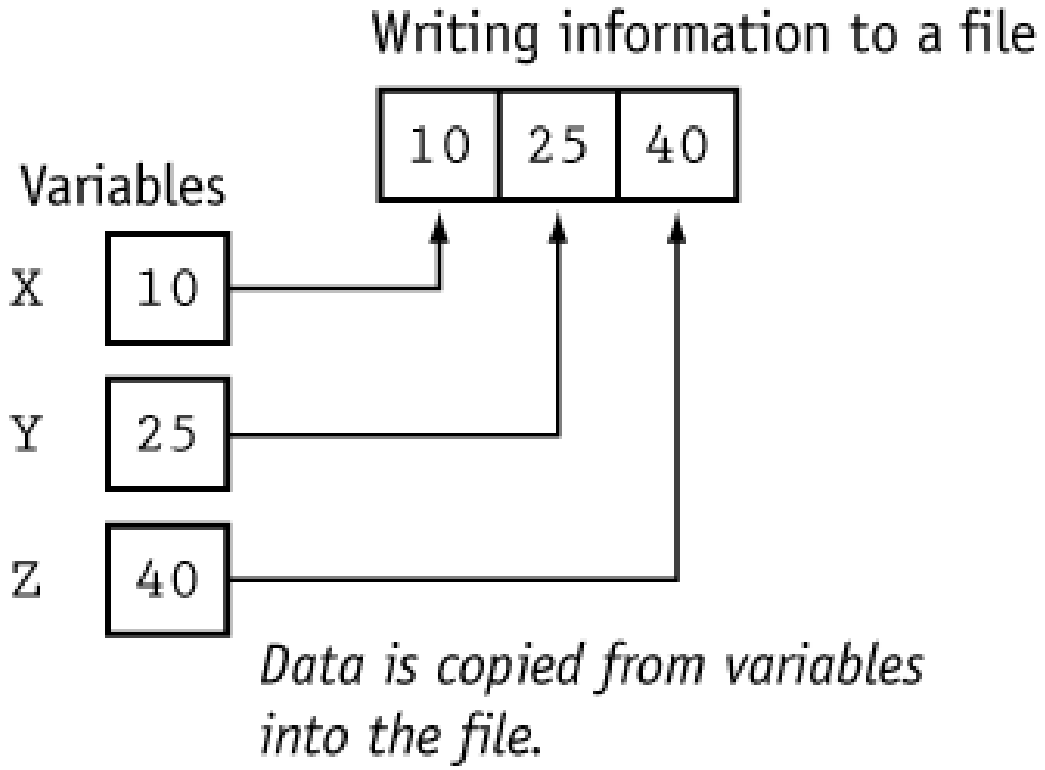
Output

```
Enter first value: 45
Enter second value: 15
Addition of two numbers: 60
Difference of two numbers: 30
Product of two numbers: 675
Division of two numbers: 3
```

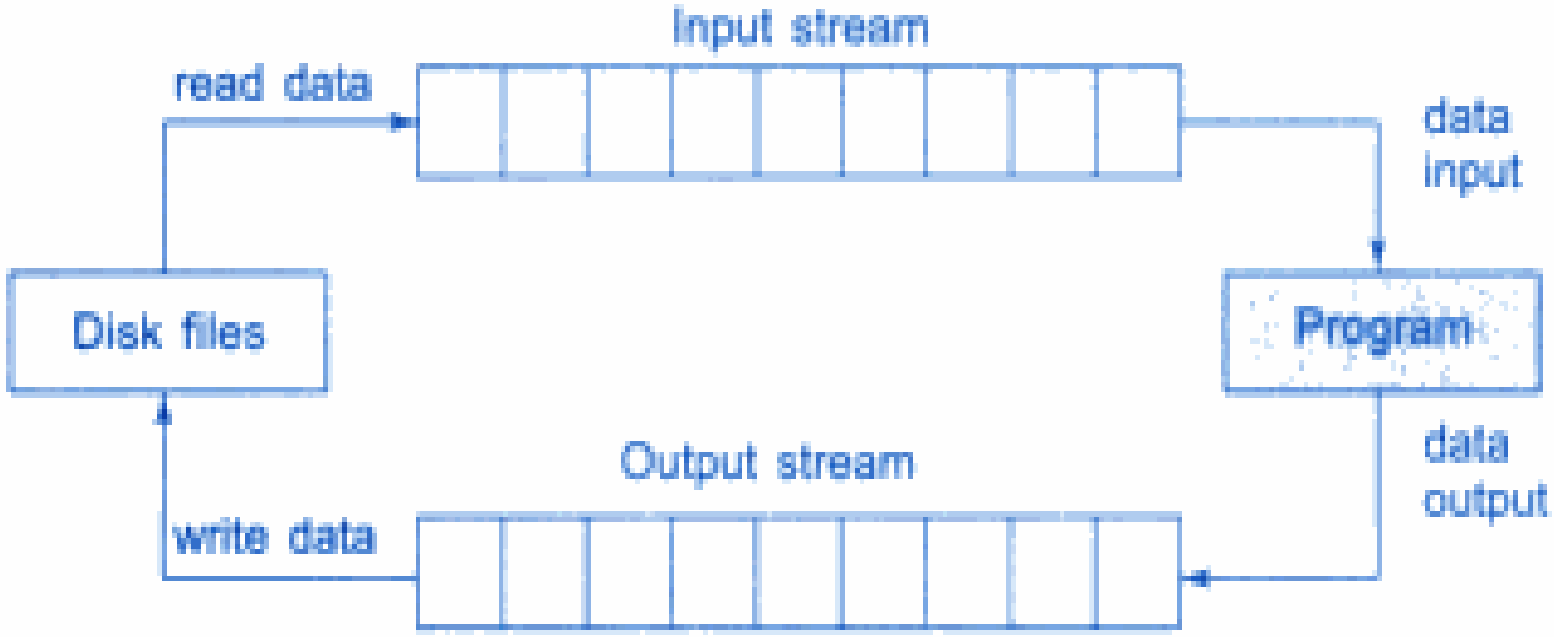
File Handling

- A file is a collection on information, usually stored on a computer's disk or information can be saved in a form of files and then later reused.
- The program data or output can be stored in these files and that persists even after program execution is over.
- The data can be read whenever necessary and can be placed back into the file after modification.
- The data remains safe provided storage media does not crash or corrupt.
- All files are assigned a name that is used for identification purposes by the operating system and the user.

File Handling



File Handling



File Handling

File Name and Extension	File Contents
MYPROG.BAS	BASIC program
MENU.BAT	DOS Batch File
INSTALL.DOC	Documentation File
CRUNCH.EXE	Executable File
BOB.HTML	HTML (Hypertext Markup Language) File
3DMODEL.JAVA	Java program or applet
INVENT.OBJ	Object File
PROG1.PRJ	Borland C++ Project File
ANSI.SYS	System Device Driver
README.TXT	Text File

Characteristic of File

- A computer file:

➢is stored on a secondary storage device (e.g., disk)

➢is permanent

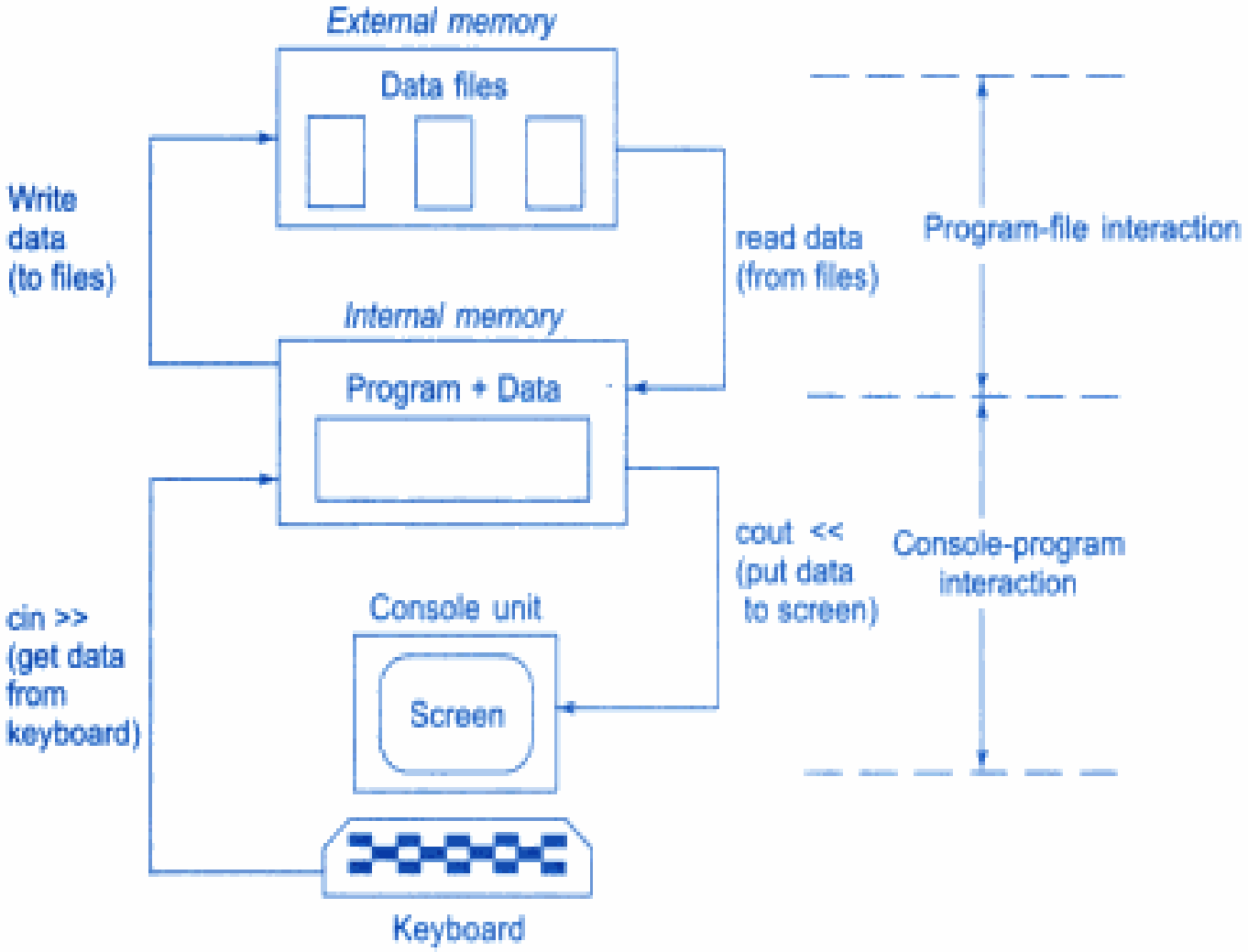
➢can be used to

- ✓provide input data to a program
- ✓or receive output data from a program
- ✓or both;

➢ should reside in Project directory for easy access

➢ Must be opened before it is used.

File Handling



Consol-program file interaction

General File I/O Steps

- Include the header file fstream in the program.
- Declare file stream variables.
- Associate the file stream variables with the input/output sources.
- Open the file
- Use the file stream variables with >>, <<, or other input/output functions.
- Close the file.

File Handling

- A stream is an abstraction that represents a device on which operations of input and output are performed.

- A stream can be represented as a source or destination of characters of indefinite length depending on its usage.

- In C++ we have a set of file handling methods.

➢ ifstream

➢ ofstream

➢ fstream.

Using Input/Output Files

- **iostream -** a sequence of characters

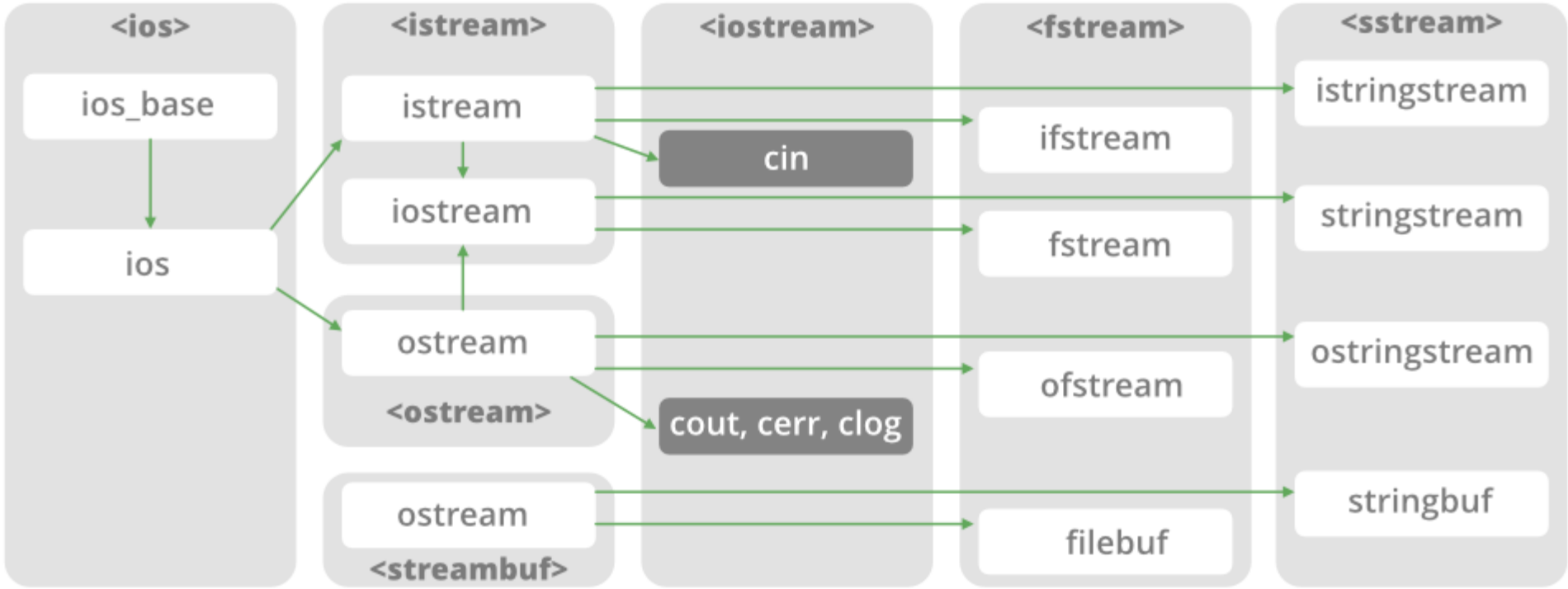
  - interactive (iostream)
    - **cin -** input stream associated with **keyboard.**
    - **cout -** output stream associated with **display**

  - file (fstream)
    - **ifstream -** defines new input stream (normally associated with a file).
    - **ofstream -** defines new output stream (normally associated with a file).

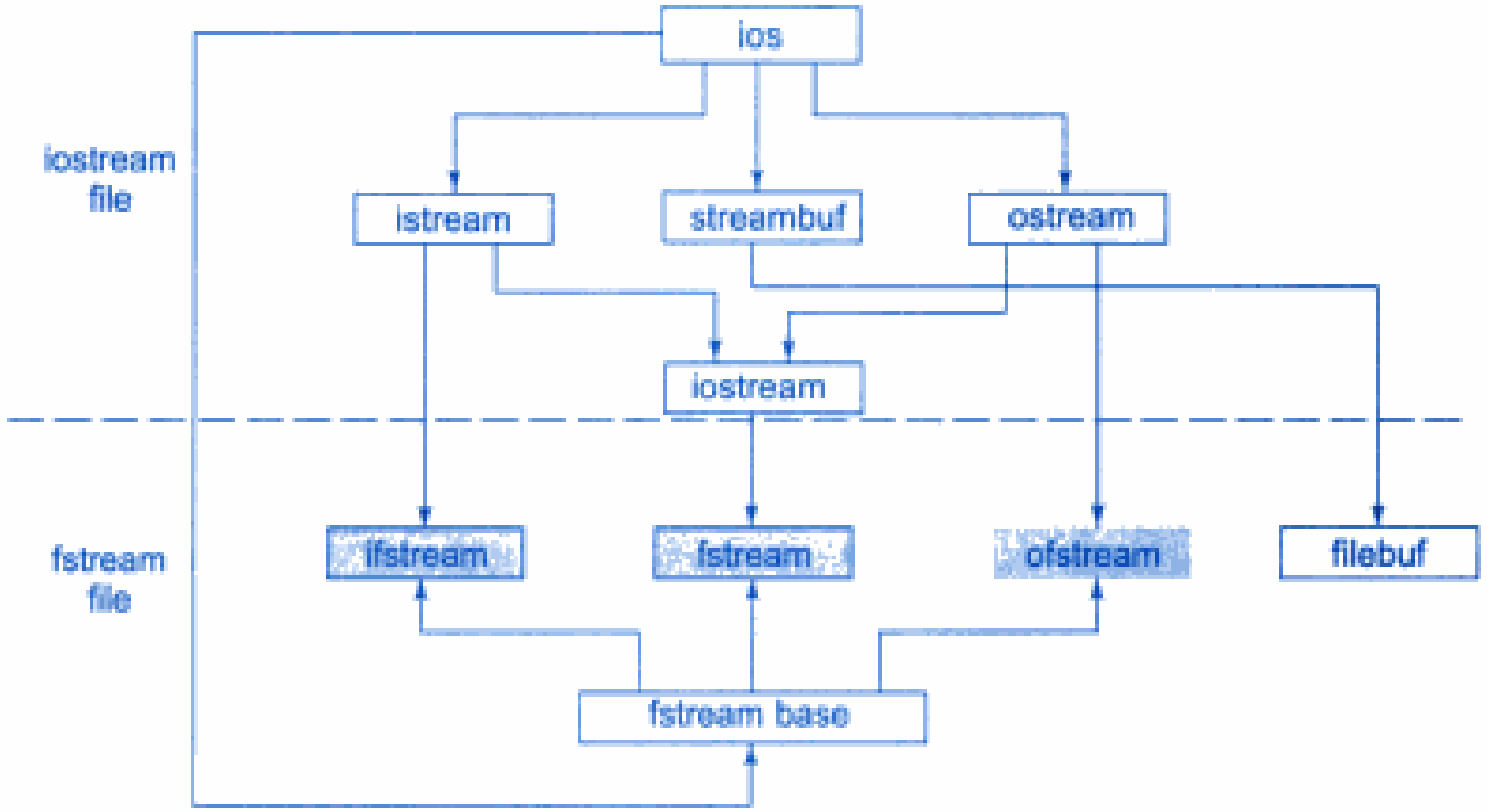
Stream I/O Library Header Files

Data Type	Description
ofstream	This data type represents the output file stream and is used to create files and to write information to files.
ifstream	This data type represents the input file stream and is used to read information from files.
fstream	This data type represents the file stream generally, and has the capabilities of both ofstream and ifstream which means it can create files, write information to files, and read information from files.

Stream I/O Library Header Files



Stream I/O Library Header Files



C++ streams

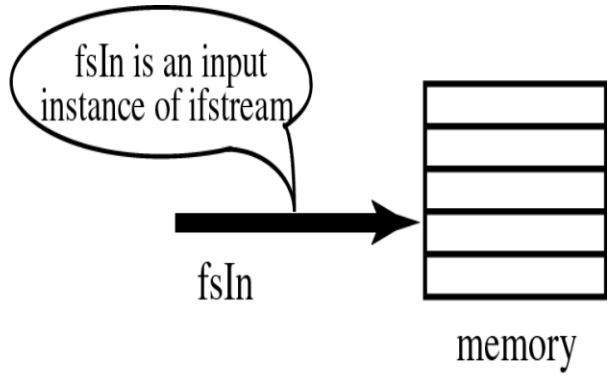
```
//Add additional header files you use
#include <fstream>
int main ()
{ /* Declare file stream variables such as
 the following  */
ifstream  fsIn;//input
ofstream fsOut; // output
fstream both //input & output
//Open the files
fsIn.open("prog1.txt"); //open the input file
fsOut.open("prog2.txt"); //open the output file
//Code for data manipulation
.
.
//Close files
fsIn.close();
fsOut.close();
return 0; }
```

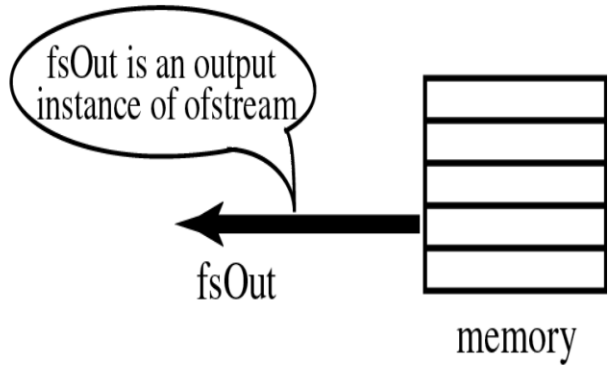
```
#include <fstream.h>

int main (void)
{
// Local Declarations
   ifstream      fsIn;

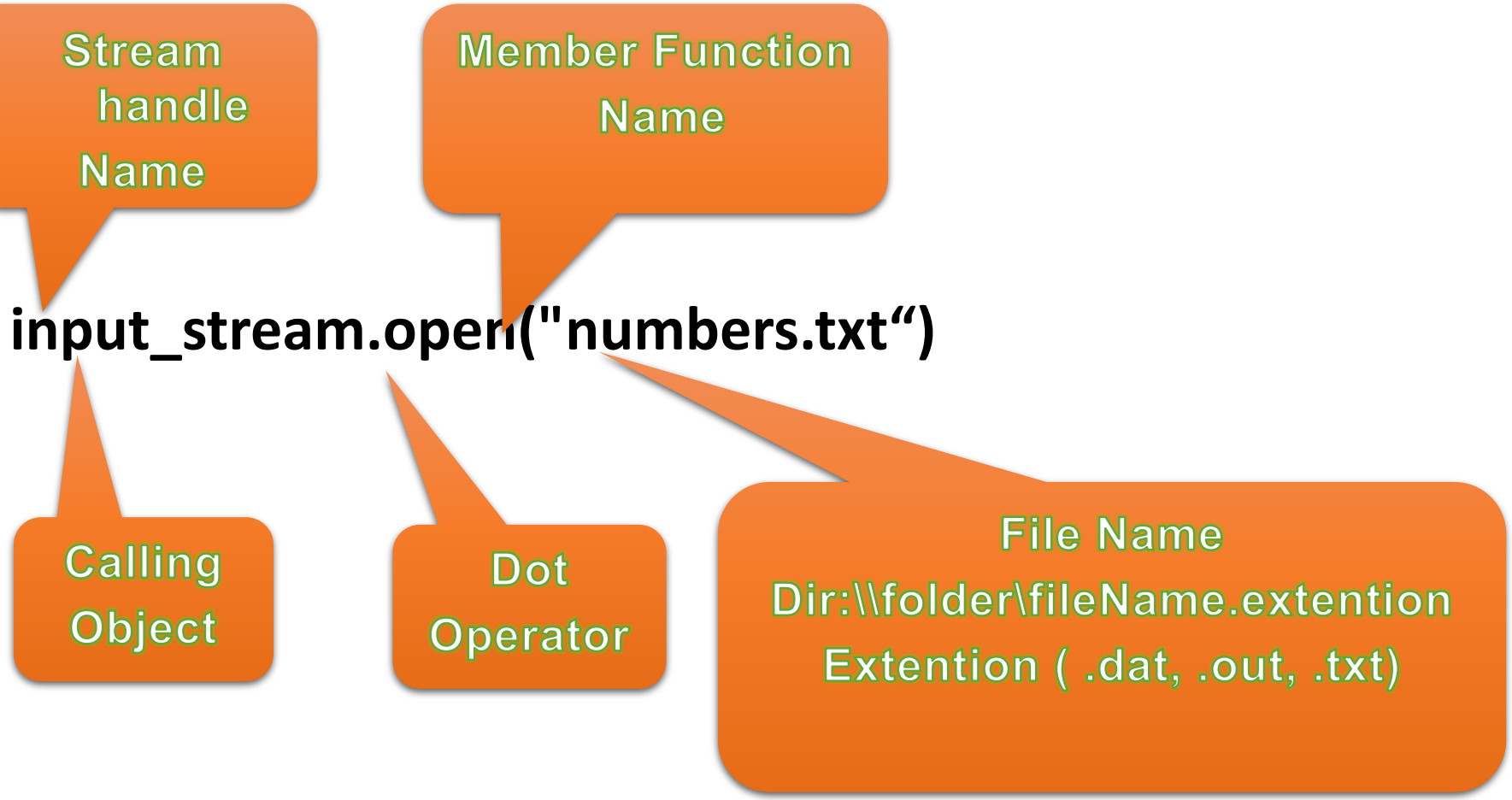
   ofstream      fsOut;
   •
   •
   •

}  // main
```





Object and Member Functions



C++ Files and Streams

- C++ provides us with the following operations in File Handling.

➢Creating a file: open()
➢Reading data: read()
➢Writing new data: write()
➢Closing a file: close()

Functions use in File Handling

Function	Operation
open()	To create a file
close()	To close an existing file
get()	Read a single character from a file
put()	write a single character in file.
read()	Read data from file
write()	Write data into file.

Opening a File

- Opening a File provides us with the following operations in File.

- The first operation performed on an object of one of these classes is to associate it to a real file. This procedure is known to open a file.

- A file using any one of the following methods:

➢First is bypassing the file name in constructor at the time of object creation.

➢Second is using the open() function.

- To open a file use open() function

Syntax

void open(const char* file_name, ios::openmode mode);

Default Open Modes

Syntax

void open(const char* file_name,ios::openmode mode);

- The first argument of the open function defines the name and format of the file with the address of the file.

- The second argument represents the mode in which the file has to be opened. The following modes are used as per the requirements.

Default Open Modes

- ifstream ios::in
- ofstream ios::out
- fstream ios::in | ios::out

new_file.open(“new_file.txt”, ios::out | ios::app);

- input mode and append mode are combined which represents the file is opened for writing and appending the outputs at the end.

Opening a File modes

Modes	Description
in	Opens the file to read(default for ifstream)
out	Opens the file to write(default for ofstream)
binary	Opens the file in binary mode
app	Opens the file and appends all the outputs at the end
ate	Opens the file and moves the control to the end of the file
trunc	Removes the data in the existing file
nocreate	Opens the file only if it already exists

<< and >> Operator

<< Insertion Operator

- for Writing purposes from RAM to HDD

>> Extraction Operator

- for Reading purposes from HDD to RAM

Example of Reading from a File

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
fstream new_file;
new_file.open("new_file_write.txt",ios::in);
if(! new_file)
{
cout<<"No such file";
}
else
{
char ch;
while (new_file.eof()==0)
{
new_file >> ch;
cout << ch;
}
new_file.close();
return 0;
}
```

Example of Reading from a File

new_file_write.txt

ACBDUFHJKHJJGGVB !

ACBD…….B

Example of Writing to a File

```
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
fstream new_file;
new_file.open("new_file_write.txt",ios::out);
if(!new_file)
{
cout<<"File creation failed";
}
else
{
cout<<"New file created“<<endl;
new_file<<"Learning File handling";
new_file << “I am done”;
    //Writing to file
new_file.close();
}
return 0;
}
```

Example of Writing to a File

new_file_write.txt



Example of reading and writing on same file

```
#include <iostream.h>
#include <fstream.h>
void main( )
{
fstream rw;
rw.open(“demo1.txt”,ios : :out);
char str1[50];
Char str2[50];
cout<<“ENTER A STRING”<<endl;
cin.getline(str1,50);
rw<<str<<endl;
rw.close( );
rw.open(“demo1.txt”,ios : :in);
rw.getline(str2,50);
cout<<“STRING READ FROM FILE”<<endl;
cout<<str2<<endl;
rw.close( );
}
```

Example of Reading and writing mobile detail

```
#include <iostream.h>
#include <fstream.h>
void main( )
{
fstream rw;
rw.open("demo2.txt",ios : :out);
char mcomp[20],model[10];
float price;
cout<<"Enter the mobile model"<<endl;
cin.getline(model, 10);
cout<<"Enter the mobile company"<<endl;
cin.getline(mcomp, 20);
cout<<"Enter the mobile price"<<endl;
cin>>price;
rw<<model<<endl<<mcomp<<endl<<price<<endl;
rw.close( );
rw.open("demo2.txt",ios : :in);
rw.getline(model,10);
rw.getline(mcomp,20);
rw>>price;
cout<<"Mobile details read from file"<<endl;
cout<<"Model :="<<model<<endl;
cout<<"Company :="<<mcomp<<endl;
cout<<"Price :="<<price<<endl;
rw.close( );
}
```

Example of append mode

```
#include<iostream>
#include<string>
#include<fstream>
using namespace std;
int main() {
   fstream f;
   ofstream fout;
   fout.open (“demo1.txt",ios::app);
      if(!fout)
     {
         cout<<"File creation failed";
}
else
{
      fout<<Data has been appended to file<<endl;
     fout.close();
return 0;
 }
```

Reading and writing mobile detail

Output:

Data has been appended to file

Binary Mode (Writing)

- To write a binary file in C++ use the write() method.
- It is used to write a number of bytes on the given stream.
- The file is extended if the put pointer is currently at the end of the file.
- If this pointer points into the middle of the file, characters in the file are overwritten with the new data.
- Syntax of write method

ofstream& write(const char*, int);

Binary Mode (Reading)

- To read a binary file in C++, use the read() method.
- It extracts a given number of bytes from the given stream and place them into the memory, pointed to by the first parameter.
- If any error is occurred during reading in the file, the stream is placed in an error state; all future read operations will fail then.
- Syntax of read method

ifstream& read(const char*, int);

Example of binary mode

```
#include<iostream>
#include<fstream>
using namespace std;
struct Student {
  int roll_no;
  string name;
};
int main() {
  ofstream wf;
wf.open("student.dat", ios::out | ios::binary);
  if(!wf)
{
    cout << "Cannot open file!" << endl;
}
  Student wstu[3];
  wstu[0].roll_no = 1;
  wstu[0].name = "Ram";
  wstu[1].roll_no = 2;
  wstu[1].name = "Shyam";
  wstu[2].roll_no = 3;
  wstu[2].name = "Madhu";
  for(int i = 0; i < 3; i++)
{
wf.write((char *) &wstu[i], sizeof(Student));
}
wf.close();
ifstream rf;
rf.("student.dat", ios::out | ios::binary);
  if(!rf)
{
   cout << "Cannot open file!" << endl;
}
```

Example of binary mode

```
Student rstu[3];
   for(int i = 0; i < 3; i++)
{
rf.read((char *) &rstu[i], sizeof(Student));
}
   rf.close();
   cout<<"Student's Details:"<<endl;
   for(int i=0; i < 3; i++)
{
      cout << "Roll No: " << wstu[i].roll_no << endl;
      cout << "Name: " << wstu[i].name << endl;
      cout << endl;
   }
   return 0;
}
```

Reading and writing mobile detail

Output:

Student's Details:

Roll No: 1

Name: Ram

Roll No: 2

Name: Shyam

Roll No: 3

Name: Madhu

Example of file copying using get() and put()

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>
void main( )
{
fstream source, dest;
char sfile[15], dfile[15];
char ch;
cout<<“Enter the source file name”<<endl;
cin>>sfile;
source.open (sfile, ios : :in);
cout<<“Enter the destination file name”<<endl;
cin>>dfile;
dest.open (dfile,ios : :out);
while(source.eof( )==0)
{
source.get(ch);
dest.put(ch);
}
source.close( );
dest.close( );
}
```

Output of file copying using get() and put()

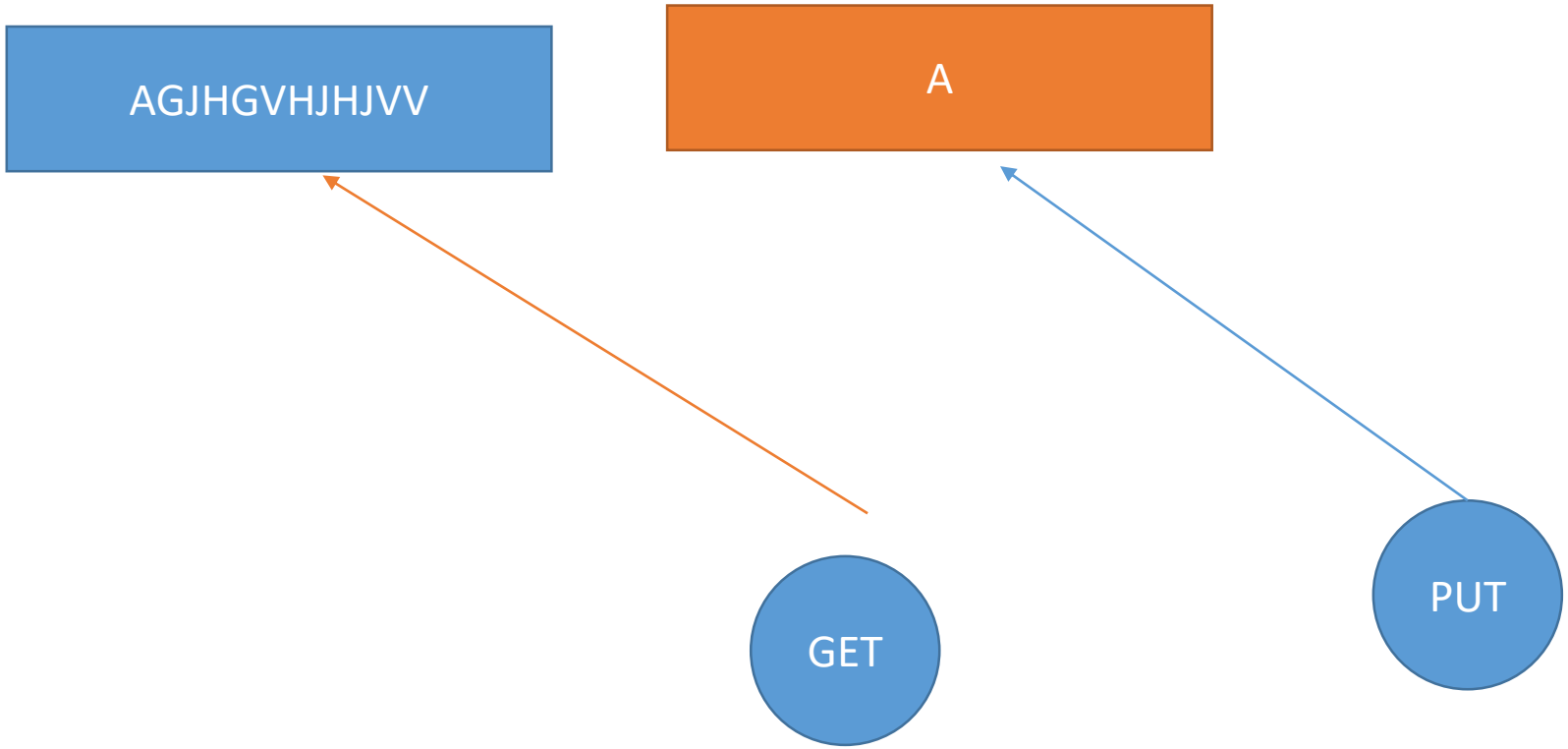
OUTPUT ?

Enter the source file name

Sample.txt

Enter the destination file name

Sample 2.txt



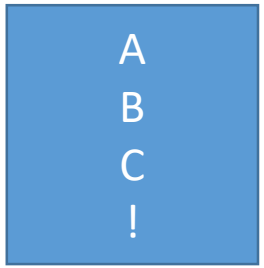
Example of working with two files at a time

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>
void main( )
{
fstream f1, f2;
f1.open(“names.txt”,ios : :out);
f1<<“A”<<endl;
f1<<“B”<<endl;
f1<<“C”<<endl;
f1.close( );
f2.open(“sname.txt”,ios : :out);
f2 <<“D”<<endl;
f2<<“E”<<endl;
f2<<“F”<<endl;
f2.close( );
```

```
f1.open(“names.txt”,ios : :in);
f2.open(“sname.txt”,ios : :in);
char str[15];
while(f1.eof( )==0)
{
f1.getline(str,15);
cout<<str<<“ “<<endl;
f2.getline(str,15);
cout<<str<<endl;
}
f1.close( );
f2.close();
}
```

Example of working with two files at a time

names.txt



snames.txt



Random Access in File

- Random access means reading data randomly from any where in the file.
- For this purpose need to set the position of the file pointer first in the file and then read the data.
- C++ file stream classes provides functions for the manipulation of file pointer any where in the file.

Random Access in File

- Following functions can access data in random fashion.

➢Seekg( )

➢seekp( )

➢tellg( )

➢tellp( )

Manipulation Function

- The function seekg() and tellg() are used when reading from file i.e., when used with the get pointer that's why the suffix g.
- The function tellp() and seekp() are used when writing to the file i.e., when put pointer is used that's why the suffix p.
- The seek (p/g) function moves pointer to the specified position and tell (p/g) gives the position of the current pointer.
- The most frequently used function is seekg whose syntax is as:

seekg (streamoff off, ios : :seek_dir dir);

Random Access in File

- The **off** is the new offset value of **streamoff** class.
- **Offset positive** means move forwards, −ve means move backwards and 0 means stay at the current position.
- First byte in the file is at position 0. The dir is the seek diretion.
- Must be done of the following enumerators :

→ ios : :beg Seek from the beginning of the stream.

→ ios : :cur Seek from the current position in the stream.

→ ios : : end Seek from the end of the stream.

Example

ifstream file;

- file.seekg(3,ios : :beg)-> Third byte from the beginning.
- file.seekg(6,ios : :cur)-> Forward by 6 bytes from current pos.
- file.seekg(-5,ios : :cur)->Backward by 5 bytes from current pos.
- file.seekg(-4,ios : :end)-> Backward by 4 bytes from end.
- file.seekg(0,ios : :cur)-> stay at the current position.

Random Access in File

- Similar, to seekg() can use seekp() for moving pointer within a file which is opened for writing.
- The function tellg() and tellp() can be used for finding number of bytes into the file by opening the file and moving pointer to the end of the file.

```
ofstream file("num.txt", ios : :ate);

cout<<file.tellp( )<<endl;
```

Templates

Templates

- Templates are the foundation of generic programming, which involves writing code in a way that is independent of any particular type.

- A template is a blueprint or formula for creating a generic class or a function.

- The library containers like iterators and algorithms are examples of generic programming and have been developed using template concept.

Templates

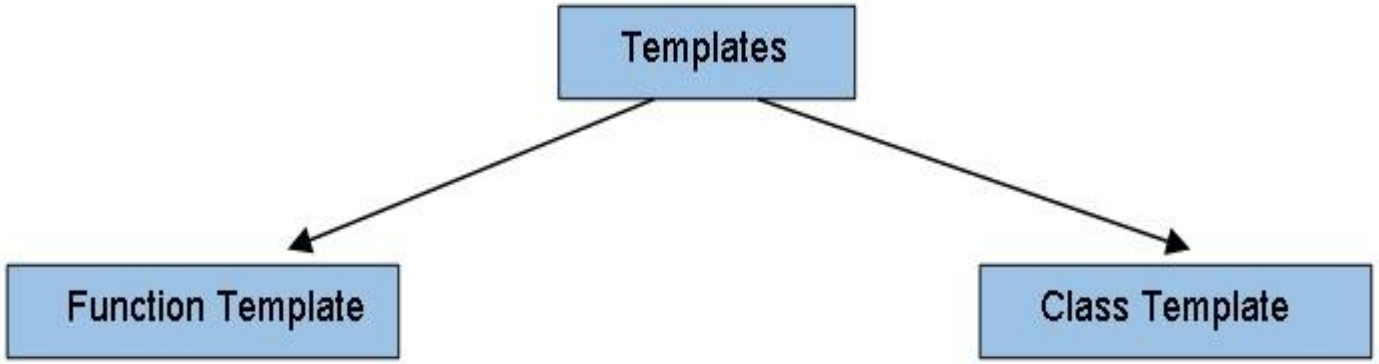
- Templates
  - ➢ Easily create a large range of related functions or classes
  - ➢ Function template - the blueprint of the related functions
  - ➢ Template function - a specific function made from a function template

- Templates make it possible to use one function or class to handle many different data types.

- The template concept can be used in two different ways: with functions and with classes.

Templates

- We have mainly two kinds of templates:
- Function Templates
- Class Templates



Function Templates

```
template <typename T>
T myMax(T x, T y)
{
    return (x > y)? x: y;
}

int main()
{
   cout << myMax<int>(3, 7) << endl;
   cout << myMax<char>('g', 'e') << endl;
   return 0;
}
```

Compiler internally generates and adds below code

```
int myMax(int x, int y)
{
    return (x > y)? x: y;
}
```

Compiler internally generates and adds below code.

```
char myMax(char x, char y)
{
    return (x > y)? x: y;
}
```

Function Templates

- The general form of a template function definition is shown here:

```
template <class type> ret-type func-name(parameter list)
{
// body of function
}
```

Example 1: Function Template

```
#include <iostream>
using namespace std;
template <typename T> T myMax(T x, T y)
{
        return (x > y) ? x : y;
}

int main()
{

cout << myMax<int>(3, 7) << endl;

cout << myMax<double>(3.0, 7.0);

cout << myMax<char>('g', 'e');
        return 0;
}
```

Output

Output :

7

7.0

g

Class Templates

- Class template has similar syntax as function templates.
- It can be defined as more than one generic data type by using a comma-separated list.
- The general form of a generic class declaration is shown here:

```
template <class type> class class-name
{
.
.
.
}
```

Example 1

```
#include <iostream>
 using namespace std;
 template <class T1, class T2>
class A
{
    T1 a;
    T2 b;
    public:
   A(T1 x,T2 y)
   {
      a = x;
      b = y;
   }
void display()
{
cout << "Values of a and b are : " << a<<" ,"<<b<<;
 }
 };
  int main()
{
        A< int, float > d (5, 6.5);
        d.display();
        return 0;
}
```

Output

Output :

Values of a and b are : 5,6.5

Example 2: Class Template

```
#include <iostream>
using namespace std;
template<class T, int size>
class A
{
   public:
   T arr [size];
   void insert()
   {
      int i =1;
      for (int j=0;j<size;j++)
      {
         arr[j] = i;
         i++;
      }
   }
   void display()
   {
      for(int i=0;i<size;i++)
      {
         cout << arr[i] << " ";
      }
   }
};
int main()
{
   A<int,10> t1;
   t1.insert();
   t1.display();
   return 0;
}
```

Output

Output :

1 2 3 4 5 6 7 8 9 10

Templates and friends

- Friendships allowed between a class template and

➢global function

➢member function of another class

➢entire class

Templates and friends

- Friend functions: inside definition of class template X:

friend void f1();

f1() a friend of all template classes

Templates and friends

friend void f2(X< T > &);

f2(X< int > &) is a friend of X < int > only.

The same applies to float, double, etc.

Templates and friends

friend void A::f3();

- Member function f3 of class A is a friend of all template classes.

Templates and static Members

- **Non-template class:**

➢static data members shared between all objects

- **Template classes:**

➢each class (int, float, etc.) has its own copy of static data members

➢static variables initialized at file scope

➢Each template class gets its own copy of static member function

End of Unit 4

THANKS…..